
◆ SPLIT DATA

```
X_train, X_test, y_train, y_test =
train_test_split(
    X_vectorized,
    y,
    test_size=0.2,
    random_state=42
)
```

`train_test_split()` → sklearn function

`X_vectorized` → input features (numbers from TF-IDF)

`y` → labels (0 = HAM, 1 = SPAM)

`test_size=0.2` → 20% data for testing

`random_state=42` → fixed random split

👉 Use: divide dataset into training and testing sets.

◆ WHAT IS X AND Y?

Before split:

```
X = email messages (converted to numbers using TF-IDF)
y = labels (0 or 1)
```

Example:

```
X → features
[0.2, 0.8, 0.1]
[0.9, 0.1, 0.4]
...

y → labels
1
0
1
```

◆ WHAT DOES train_test_split DO?

It shuffles and divides data into two parts:

```
TRAINING SET → for learning
TESTING SET → for checking performance
```

◆ OUTPUT VARIABLES

```
X_train → training features
X_test  → testing features
y_train → training labels
y_test  → testing labels
```

◆ SIMPLE EXAMPLE

Suppose we have 10 emails:

```
Email 1 → SPAM
Email 2 → HAM
Email 3 → SPAM
...
Email 10 → HAM
```

After split:

```
TRAINING (80%)
Email 1 → SPAM
Email 2 → HAM
...
Email 8 → SPAM

TESTING (20%)
Email 9 → HAM
Email 10 → SPAM
```

◆ WHAT IS 80–20 SPLIT?

```
80% → training data (model learns patter
20% → testing data (model is evaluated)
```

Example:

```
1000 emails
↓
800 training
200 testing
```

◆ WHY DO WE NEED IT?

If you train and test on same data:

```
Model will just memorize answers ❌
```

Instead:

```
Train on one part
Test on unseen data
```

This checks real performance.

◆ WHAT IS random_state=42? // check random 42

This controls randomness.

Without it:

```
Every run gives different split
```

With it:

```
Same split every time
```

`42` is just a common fixed number (no special meaning).

◆ COMPLETE FLOW

```
Messages → TF-IDF → Numbers (X_vectorized)
↓
train_test_split
↓
Training Set → model learns
Testing Set → model is checked
```

◆ SUMMARY

```
train_test_split → splits dataset into training and testing parts

X_train → features used to train model

X_test → features used to test model

y_train → correct answers for training

y_test → correct answers for testing

test_size=0.2 → 20% data reserved for testing

random_state=42 → ensures same split every time
```

👉 Use: ensure model is trained on one set of data and tested on unseen data for accurate evaluation.